דף סיכום בחינה

מזהה סטודנט: N102114802

מזהה קורס: 20594   שם קורס: מערכות הפעלה

מספר מבחן: 0001

מספר שאלה בטופס בחינה זה	ניקוד מירבי	ציון	התשובה שסומנה	התשובה הנכונה	מיקום השאלה בטופס המקור	מיקום התשובה שסומנה בטופס המקור
1	25.00	25.00	-	-	1	-
2	19.00	19.00	-	-	2	-
3	11.00	10.00	-	-	3	-
4	5.00	4.00	-	-	4	-
5	5.00	5.00	-	-	5	-
6	5.00	3.00	-	-	6	-
7	5.00	5.00	4	4	7	4
8	5.00	0.00	2	1	8	2
9	5.00	5.00	4	4	9	4
10	5.00	5.00	2	2	10	2
11	5.00	5.00	4	4	11	4
12	5.00	5.00	2	2	12	2

כמות תשובות נכונות (עבור שאלות סגורות) : 5

ציון בחינה סופי : 91.00

מקרא צבעים

תשובה שגויה
שאלה שלא נענתה
שאלה מבוטלת

הבחינה הבדוקה בעמודים הבאים

דף תשובות

האוניברסיטה הפתוחה

מספר גרסה	מספר תעודת זהות	מספר הקורס	תאריך בחינה
0001	213783681	20594	270623

הוראות למילוי דף התשובות

1. נא למלא תשובה אחת בלבד על ידי השחרת העיגול - ●
2. את ההשחרה יש לבצע בעט שחור או כחול בלבד!
3. השימוש בטיפקס או מרקרים אסור!
4. כדי לתקן סימון, יש לסמן X על התשובה הלא נכונה - ולהשחיר את התשובה הנכונה - ●
5. כדי לחזור לסימון המקורי, יש לסמן X על התשובה הלא רצויה, ולהשחיר מחדש את סימון ה- באופן שיכסה המחיקה -

תשובה \ שאלה	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
א																				
ב								●		●		●								
ג																				
ד							●		●		●									
ה																				
ו																				

תשובה \ שאלה	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
א																				
ב																				
ג																				
ד																				
ה																				
ו																				

תשובה \ שאלה	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
א																				
ב																				
ג																				
ד																				
ה																				
ו																				

שאלון 517 82.8f.7 M4

2|1|3|7|8|3|6|8|1

מספר תעודת זהות
יש לרשום את כל תשע הספרות

ש מ N102114802

ת.ז: 213783681 מספר סידורי: 7

האוניברסיטה הפתוחה

ח' בתמוז תשפ"ג

27 ביוני 2023

מס' שאלון - 517

מס' מועד 82

סמסטר 2023ב

20594 / 4

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **12** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלקים א ו - ב מופיעות שאלות פתוחות. ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה. את יש לכתוב בשאלון הבחינה.

ג. בחלק ג (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה הנכונה ביותר מבין התשובות המוצעות. את התשובות בחלק ג' עליכם לסמן בטופס המחשב המצורף שבסוף השאלון.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.
אין להכניס חומר דיגיטלי!

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

שאלון 517 82.81.7 M4

אם תורי -
אז מהפונקציה CAS (turn, mynum, mynum+1)
יחזור המספר שלי:
while (CAS(turn, mynum, mynum+1) != mynum)
המתנה פעילה
turn, mynum, mynum+1)
turn = 0
= mynum+1

נניח A
מגיע המנעול 0
נכנס קודם
זמן ה while
מדלג כי ...
נניח B מגיע המנעול 1
נכנס קודם
זמן מחכה
נניח C מגיע המנעול 1
נכנס קודם
זמן מחכה
A ... turn ... 0
רק אחד מ B או C שידלג ראשון CAS יקבל המספר 0
... למשנה הדבר

הנחיות למבחן:

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלקים א׳ ו ב׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות, בכתב קריא ובקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.
2. בחלק ג׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה הנכונה ביותר מבין התשובות. **את התשובות בחלק ג׳ עליכם לסמן בטופס המחשב המצורף שבסוף השאלון.**
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (55 נקודות)

ענו על שאלות 1-3.

שאלה 1 (25 נקודות)

נתונה מערכת מחשב שבה המעבד CPU תומך בפקודה אטומית CAS – COMPARE AND SWAP.

להלן פסודוקוד המתאר את פעולת הפקודה:

```
int compare _and_swap(int *value, int expected, int new_value) {
int temp = *value;
 if (*value == expected) *value = new_value;
 return temp; }
```

א. (7 נק׳)

על בסיס הפקודה הזאות ממשו את פרוטוקול **פשוט ביותר בסיסי** לכניסה ויציאה של קטע קריטי המיועד לסנכרון תהליכונים THREADS הרצים במקביל.

הפרוטוקול חייב להבטיח את **מניעה הדדית והתקדמות**, אך **אינו צריך להבטיח המתנה מוגבלת**(העדר הרעבה). הסבירו כיצד הפרוטוקול עובד.

תשובה והסבר:

פתרון: נגדיר משתנה חדש turn מאותחל ל-0 המשותף לכל התהליכים:

int turn = 0

כל תהליך שרוצה להכנס לקטע קריטי יבצע את הפרוטוקול הבא:

```
~~enter~~ // enter
while (compare_and_swap(&turn, 0, 1) != 0)
{
  //do nothing, busy wait.
}
```

המשך בעמוד הבא

שאלון 517
82.81.7 M4

שם המנעול משוחרר?
מחכה תהליך
נניח

תנאים לפתרון קטע קריטי:

- מניעה הדדית
- התקדמות מוגבלת - התקדמות
- תהליך שנמצא מחוץ לא ימנע מאחר להכנס
- אין הסתמכות על הנחות של חומרה ונכונות

נשים לב: n=3

נכנס תהליך A - i=0

waiting[0]=true
key=1
key=CAS(lock,0,1)

אם המנעול יש 0 - ניכנס.
המנעול נתפס משוחרר. נכנס.

waiting[0]=false

נכנס תהליך i=3

waiting[3]=true
key=1

המנעול מכיל 1 - תפוס A, לכן נשאר בלולאה.

A גומר:

נכנס תהליך i=2

waiting[2]=~~false~~ true
~~wa~~ key=1

המנעול על 1 תפוס A, לכן נשאר בלולאה.

~~יוצא~~ A גומר

j=1 waiting[1] מאותחל ל-false
j=2 true = waiting[2
j≠i לכן waiting[2]=false

עכשיו נכנס תהליך 1
אחריו תהליך 0
אחריו תהליכים n-1
→ אין הגנה - 0 יכנס אחרון.
סעיף ג

כשיש 2 תהליכים
מדובר במקביל שרות
waiting ו- key=1.
אחרי תהליך אחד נכנס
ל while ומשנה את key
התהליך השני לא יכנס ללולאה!

נא לא לכתוב בשוליים

* כך לא יכולים 2 תהליכים להיכנס ביחד לקטע הקריטי - מניעה הדדית
ותהליך שרוצה מהקטע הקריטי מאפשר לתהליך הבא להיכנס אליו - התקדמות

// critical section

turn=0 //exit

הסבר: כל Thread שרוצה להיכנס יבקש את הפקודה CAS(&turn,0,1). הפקודה תשנה את turn ל-1 ותחזיר 0 אם קודם turn היה שווה 0. אם הקטע הקריטי פנוי (למשל בהתחלה) אז turn=0 ולכן התנאי ב-while לא יתקיים והתהליך יכנס לקטע הקריטי, תוך שהוא מבצע השמה של 1 ל-turn. כל תהליך נוסף שינסה להיכנס עכשיו לא יצליח כי CAS(&turn,0,1) יחזיר 1 ולכן הוא לא יצא מהלולאה. רק כאשר ה-thread הנוכחי יצא מהקטע הקריטי וידליק turn=0 ואז תהליך אחר יתפוס את turn, ישנה אותו ל-1 ויכנס לקטע הקריטי *

ב. (12 נק׳)

הוצע הפרוטוקול הבא (לא הכי פשוט) לפתרון בעיית קטע קריטי המבוסס על אותה פקודה:

entry

```
while (true) {
waiting[i] = true;
key = 1;
while (waiting[i] && key == 1) {
key = compare and swap(&lock,0,1);}
waiting[i] = false;
/* critical section */
j = (i + 1) % n;
while ((j != i) && !waiting[j]) {
j = (j + 1) % n;}
if (j == i)
lock = 0;
else
waiting[j] = false;
/* remainder section */
}
```

האם הפרוטוקול המוצע עומד בכל דרישות פתרון קטע קריטי כולל המתנה מוגבלת?

הוכיחו שכן או הפריכו ע״י דוגמא נגדית.

תשובה והסבר:

לא, משתנה key משותף לכל התהליכים (כך נראה כי הוא לא מוגדר בקטע כאן). הפרוטוקול לא עומד בדרישה למניעה הדדית. הוכחה:
נניח תהליכים A ו-B רוצים להיכנס לקטע הקריטי בו-זמנית ולכן lock=0
תהליך A מדליק את שורות 1 ו-2: waiting[iA]=T, key=1. כעת תהליך B מדליק את השורות 1 ו-2 -
waiting[iB]=T, key=1. עכשיו יש csw ושוב חוזר תהליך A. הוא נכנס ללולאת ה-while,
מדליק פקודת CAS שמחזירה 0 ולכן עכשיו key=0. הוא בודק שוב את תנאי הלולאה, כיוון ש key==0 הוא לא
נכנס שוב ללולאה ומתקדם אל הקטע הקריטי: עכשיו יש csw, תהליך B בודק את תנאי הכניסה ללולאה.
כיוון ש-A שינה את key להיות 0, התנאי key==1 לא מתקיים ותהליך B לא נכנס ללולאה וממשיך אל הקטע
הקריטי. עכשיו יש 2 התהליכים בקטע הקריטי (הוכחנו) אותו. ביחד לא יכולים שניהם, ואין מניעה הדדית.
זה קל שההשמה הראשונה והבדיקה של key==1 ב-while אינן פעולה אטומית ואין עדיין הגנה ולכן אפשר לאחר
ההשמה לעצור, וכאשר תהליך אחר מדליק key=0 לבדוק את ה-while ולהיכנס לקטע הקריטי.
שאר הדרישות מתקיימות - אין הרעבה כי תמיד יבדקו כל n התהליכים ואם תהליך מעוניין להיכנס
הוא לא יחכה לנצח אלא יקבל תור תוך n-1 תהליכים הקודמים לו לכל היותר.
אין תנאי של חומרה וכו', ואין [illegible] של תהליך שנמצא מחוץ לקטע הקריטי
ומונע מתהליך אחר להיכנס.

שאלון 517 82.81.7 M4

נא לא לכתוב בשוליים

25
(1)

ג. (6 נק')

האם הפרוטוקול המוצע מבטיח גם הוגנות(מי שניסה להיכנס ראשון, ייכנס ראשון אחרי שקטע קריטי יתפנה)? הוכיחו שכן או הפריכו ע"י דוגמא נגדית.

תשובה והסבר:

לא, הפרוטוקול לא מבטיח הוגנות. דוגמא נגדית:
יש תהליך 1 שנכנס ראשון. הוא תופס את המנעול (מבצע CAS שמשנה 1 ב lock)
ונכנס לקטע הקריטי. אחריו מגיע תהליך 0. המנעול תפוס ולכן הוא לא יוצא מהלולאת while.
אחריו נכנס תהליך 2. גם הוא מחכה ב while. כאשר תהליך 1 גומר, הוא מעלה j=i+1=1+1=2
ובודק את waiting[2]. כיוון שתהליך 2 מחכה להיכנס, waiting[2]=T ולכן מתבצע waiting[2]=f
ותהליך 2 נכנס לקטע הקריטי. תהליך 0, שהגיע לפני תהליך 2, יצטרך להמתין עד שתהליך 2
וכל תהליך אחר שיכנס לימינו ומספרו יהיה גדול מ 2 יגמור, ורק בסוף הסבב יגיע תורו.
לכן אין כאן הוגנות

המשך הבחינה בעמוד הבא

שאלון 517 82.81.7 M4

כתובת וירטואלית שמתחילה ב0. ההיסט שלה קטן יותר.

מה זה אומר? גודל שמס׳ המסגרת הפיזי— איזה תמונה

יהיה נמוך.

יתרון - הקטנתי חריף היגמש.

חסרון- צריך הפרדת דפים מיוחדת

- סרקול של MMU? דפים רק

- אבל אולי [illegible] [illegible]

- תהליך [illegible] [illegible] שלו את הדרישה ?

—

שאלה 2 (19 נקודות)

נתונה מערכת מחשב שתומכת בדפים בגדלים שונים: 4 Kbyte ו 4 Mbyte בלבד. כל דף בכל גודל יכול להימצא בכל מקום בזיכרון הפיזי RAM. תהליך שמבקש הקצאת זיכרון ראשי 8 Mbyte או יותר מקבל אותו בדפים בגודל 4 Mbyte ובהקצאות קטנות יותר בדפים בגודל 4 Kbyte. המרחב הווירטואלי מחולק לשני אזורים שווים, בחלק הנמוך גודל הדף 4 Kbyte ובחלק הגבוה 4 Mbyte **במרחב הזיכרון הפיזי אין חלוקה כזאת.**

הקומפיילר המיוחד שפותח בשביל המערכת ופונקציות הספרייה של שפת תכנות קובעים כתובות וירטואליות של משתנים בהתאם(בכתובות גבוהות או נמוכות, לא משנה מה הדרך לעשות את זה).

א. (9 נק')

תארו את האלגוריתם(את הדרך) של MMU לתרגום הכתובת הוירטואלית לכתובת פיזית. ציינו האם נדרשים שינויים בצורת הכתובת הוירטואלית ובטבלת הדפים(בהשוואה לצורה רגילה) כדי לאפשר את תרגום הכתובת ואם כן, מה הם. הדגישו במיוחד את שיטת חישוב מספר מסגרת זיכרון פיזית ואת הגודל שלה.

כאשר כתובת וירטואלית מגיעה ל MMU, הדבר הראשון צריך לדעת מהו הדף, היא צריכה להיות ממופה לדף בגודל 4KB או 8KB.
לכן נסתכל על ה MSB: אם הוא 0, אז הכתובת שייכת לחלק הנמוך במרחב הוירטואלי, עם גודל דף 4KB.
אם הביט הוא 1, אז הכתובת שייכת לחלק הגבוה במרחב הוירטואלי, עם גודל דף 4MB.
בהתאם ל MSB ה MMU יקבע את גודל האופסט: אם ה MSB = 0 אז האופסט (offset) יהיה 12 סיביות (כי 4KB = 2^{12}) ואם ה MSB = 1 אז גודל ה offset יהיה 22 סיביות (כי 4MB = 2^{22}).
עכשיו צריך למצוא את מס' הדף ולגשת לטבלת הדפים כדי למצוא את מספר המסגרת בזכרון הפיזית.
בשביל שלב זה נחזיק 2 טבלאות דפים, אחת לכל חצי של הזכרון. בראשונה גודל כל שורה יהיה 12 סיביות + מידע נוסף ובשניה גודל השורה יהיה 22 סיביות + מידע נוסף. מס' השורות בטבלה הראשונה גדול יותר מהטבלה השניה כי יש יותר דפים בגודל קטן מאשר דפים גדולים. ה MMU יגש לטבלה 1 אם ה MSB הוא 0, ולטבלה 2 אם ה MSB היה 1, והאינדקס בטבלה יתאים לכל הסיביות בכתובת הוירטואלית שנמצאות משמאל ל offset, חוץ מה MSB שלא רלוונטי (רק סימן את גודל הדף וסוג הטבלה) [illegible] אם נחשב אותה נקבל עבור הדפים הגדולים אינדקסים גדולים תמיד ולכן נתאים לאינדקסים בטבלה המתאימים מ-0).

ב. (5 נק')

מה החיסרון העיקרי, חוץ מקיטוע פנימי, ויתרון העיקרי בשימוש בדפים בגודל משתנה בצורה המתוארת בשאלה?

תשובה והסבר:

היתרון העיקרי - התאמה טובה יותר בין גודל התהליך לגודל הדפים שיוקצו לו. אם תהליך גדול יוכל להשתמש בדפים גדולים הוא יצטרך פחות דפים, וכך יהיו פחות pagefault, ופחות זמן יתבזבז על חיפוש והבאת דפים אל הזכרון.

חסרון - אי אפשר לקבוע הקצאה רציפה של מסגרות זכרון בשביל תהליך. זאת כיוון שאם תהליך צריך X מסגרות זכרון בגודל 4KB, יכול להיות שאין בזכרון הפיזי רצף כזה של מסגרות קטנות הרי מעורבבות בו מסגרות גדולות וקטנות. לכן כל דף של התהליך ימופה למסגרת שנמצאת במקום אחר ואם נרצה לשפר זאת נצטרך [illegible].
במצב [illegible] חייבים פה לדעת מראש מה הדרישה של תהליך לזכרון, אלא רק כדי ריצה להקצות [illegible]
חסרונות נוספים: צריך להחזיק 2 טבלאות דפים נפרדות - סירבול ואולי לבזבז מקום רב של הטבלה [illegible]

5

שאלון 517
82.81.7 M4

נא לא לכתוב בשוליים

ג. (5 נק')

הציעו פתרון לחסרון שצוין בסעיף ב' .

תשובה והסבר :

פתרון יכול להיות חלוקה של הקטגוריה לשני חלקים שונים - של מסגרת קטנה בצורה ומסגרת
קטנה ~~[illegible]~~ ממש בחלוקה לפי רמות גדולות ומשוכות. כך נמנע חלוקה קבועה וקבועה של מיקום
כפלים למסגרת נסיון פיתור (רק נפס"ד בקו שהכיסוי המיטבי גדול וגדול, כי אז "לדחוק" מסגרת קטנה
בין מסגרות גדולות). אופציה נוספת הייתה לאפשר גמישות בגודל של הגבוה - זאת להחליט
מבחוץ אם אזור מסוים הוא מיועד למסגרת של צורה או ממש, אלא אם יש צורך בהקצאת X
מסגרות קטנות רבות - עבור לכן מקום, והאמא ודמיון לעבה בם מרוח למסגרת גדולה (שישמור השוויון).
או "שתי" מסגרות קטנות = מסגרת גדולה וכדומה.
אם לא יהיה מספיק מקום בגדל התהליך אפשר והתחל עם מסגרת קטנה ואם לא גדל הוצע להגדיר הס
למסגרת גדולה. (תגובה גבוהה מזוז). כך יפתרו את החסרון שצוין למעלה.

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים

שאלה 3 (11 נקודות)

א. (5 נקי)

הסבירו בקצרה את המטרה העיקרית ואת אופן פעולתו של רכיב ה- DMA במערכת מחשב מודרנית.

תשובה והסבר :

רכיב ה DMA הוא רכיב שתפקידו לתת גישה ישירה לזכרון לרכיבי קלט/פלט, ובכך לשחרר את המעבד לפעולות אחרות ולהעלות את רמת המקביליות של המערכת. הוא מקבל מה MMU כתובות זכרון פיזיות וגם את ההרשאה לביצוע מהרכיב הקלט/פלט, נועל את האפיק הרלוונטי בזכרון ומבצע ישירות קריאה/כתיבה לזכרון. בסיום, הוא משחרר את נעילת הזכרון.

ב. (6 נקי)

מה הן החסרונות של שימוש ב רכיב ה-DMA ? האם הוא יכול לגרום להאטת ביצוע פקודות חישוביות?

מה הן הדרכים להתמודדות עם החסרונות(ציינו את הקיימות או הציעו חדשות)?

10
(3)

תשובה והסבר :

נעילת דפים

גם תופס BUS

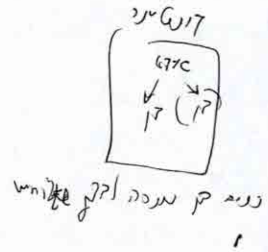
ה DMA נועל את הזכרון. בזמן הזה, המעבד לא יכול לגשת אל האפיק הרלוונטי בזכרון אם הוא צריך אותו לביצוע פעולה אחרת ולכן ה DMA עלול לגרום להאטה בביצועי ה cpu. פעולות חישוביות שאינן מצריכות גישה לזכרון יתבצעו במהירות הרגילה של ה cpu ללא האטה. כיוון ה DMA לפעמים איטי יותר מה cpu (אם המעבד מהיר מאוד) ולכן הוא עדיף שהפעולה תבוצע ע"י המעבד ולא ה DMA כי הוא יבצע אותה מהר יותר.

יש גם חסרון אבטחתי - ל DMA יש גישה ישירה לזכרון, לכן אם התוקף יש רכיב DMA זדוני יכול לגשת לזכרון וזה בעייה. לצורך מניעת הבעיה יש להשתמש בהגנה על רכיבים מפני התקף חיצוני.

ההתמודדות תהיה ע"י נעילת החלק הספציפי בלבד בזכרון שנדרש ל DMA, ותזמון פעולות חישוביות או כאלה שאין דורשות גישה לזכרון שיבוצעו ע"י המעבד בזמן פעולת ה DMA.

המשך הבחינה בעמוד הבא

נא לא לכתוב בשולים



חלק ב (15 נקודות)

ענו על שלוש השאלות 4-6.

5 נקודות לשאלה.

שאלה 4

למה יש מספר לכל קריאת מערכת SYSTEM CALL ? האם ניתן לממש את קריאת מערכת בלי שיהיה לה מספר?

4 (4)

תשובה והסבר:

המספר נועד בשביל מערכת ההפעלה. קריאות המערכת מגדירה את השיטה למערכת ההפעלה, ועל פי מספר קריאת המערכת המערכת יודעת אילו פעולות נדרשות ממנה לבצע ממקום מיוחד (למשל רגיסטר/כתובה לקובץ). לכן מספר היה דרך יותר מובנת לממש קריאות מערכת ~~ט~~ הוא צריך להעביר למערכת את הפעולות ממש שהיא לבצע לכן צריך מזהה מנהה לקריאות המערכת שהיא "חד" ערכי לכל קריאה ועל פי כך יודע המערכת מה לבצע בהתאם למצב מיוחס.

בגלל וקטור הפסיקות

שאלה 5

צוות מתכנתים עובד על הוספת תמיכה בקונטיינרים למערכת דמוית LINUX שלא תמכה בהם לפני זה.

אחרי הוספת המנגנות יצירת הקונטיינרים(קריאות מערכת חדשות, כמו UNSHARE למשל) והגדרת המשאבים שלהם, האם יהיה צורך לשנות את קוד מקור של **קריאות מערכת** קיימות?

אם עניתם לא, תסבירו איך קריאות המערכת הקיימות ימשיכו לעבוד בצורה תקינה.

אם עניתם כן, תסבירו בצורה כללית ותנו דוגמא אלו קריאות מערכת יהיה צורך לשנות ומה השינוי העיקרי הנדרש באופן ביצוען.

הערה: התשובה הכללית מדי בסגנון "צריך להתחשב בקונטיינרים" לא תתקבל כי לא מהווה הסבר.

5 (5)

תשובה והסבר:

תהליך שנמצא בתוך קונטיינר לא מודע לכל התהליכים שנמצאים מחוץ לקונטיינר, וגם להפך - תהליך מחוץ לקונטיינר לא מודע למשאבים ולתהליכים שבתוך הקונטיינר. כדי לבצע את ההפרדה צריך להתאים קריאות מערכת למשל כאלה שניגשות למערכת הקבצים כמו read, write כדי שיוכלו לגשת רק לקבצים ששייכים לתהליך המבקש אותן - אם התהליך מחוץ לקונטיינר הוא לא יוכל לגשת לקובץ שבו תהליך מתוך קונטיינר וכו'. כלומר צריך מודעות לכך שלא לכל התהליכים עכשיו יש גישה לתיקיות וכו' אלא יש הפרדה ע"י הקונטיינרים.

82.81.7 M4 שאלון 517

נא לא לכתוב בשוליים

שאלה 6

במערכת XV6 שתומכת בקונטיינרים(אבל לא בקונטיינרים מקוננים (nested הפעיל רצף של הפקודות אחרי הרצה נקייה של המערכת:

פקודות

$ pouch start c1
Pouch: c1 starting
$ pouch connect c1
 	
tty0 connected
$ mkdir mntdir
$ mount mntdir -t cgroup
cgroup filesystem already mounted
$

א. למה פקודת mount לא הצליחה עם ההודעה שהיא "already mounted", הרי מעולם המשתמש לא הפעיל ישירות את הפקודה הזאת? על ידי מי ולמה(בשביל איזו מטרה) פקודת mount כבר הופעלה?

ב. למה בעיה זהה לא תתרחש מתי שתבוצע פקודה שיוצרת קונטיינר נוסף?

3
(6)

תשובה והסבר:

א. כאשר בוצעה הפקודה הראשונה pouch start, שיצרה את הקונטיינר, היא מאחורי הקלעים ביצעה גם mount ל cgroup, וזאת כדי ליצור לקונטיינר החדש הגבלות והרשאות ספציפיות (למשל של זמן מעבד המוקצה לו וכו'). המשתמש יכול אמנם לשנות את ההגדרות הספציפיות אבל כבר [illegible] cgroup לכן אין צורך ליצור ולעשות לה mount.

משותף לכל הקונטיינרים

ב. כאשר יוצר קונטיינר ~~מכל~~ הנ"ל לא יורש את ה cgroup הקיים, אלא מצר כלי שום דבר, לכן הפקודה pouch שוב תבצע מאחורי הקלעים mount ל cgroup כדי שלקונטיינר החדש יהיו התאמות והגבלות ספציפיות, אבל לא תהיה בעיה כי דומה בזמן היצירה לקונטיינר החדש אין cgroup קיים.

המשך הבחינה בעמוד הבא

517 שאלון
82.81.7 M4

נא לא לכתוב בשוליים

חלק ג (30 נקודות)

ענו על שש שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה **ביותר** ולהקיף בעיגול את אות התשובה שבחרתם.

שאלה 7

מערכת הקבצים של מערכת הפעלה מסוימת עושה שימוש ברשימות גישה (access control lists – ACL). נניח כי במערכת מוגדרים 3 משתמשים **U1** , **U2** , **U3** . משמעות ההרשאות : RWX היא הרשאות גישה **R**ead, **W**irite, **E**execute . להלן רשימת ACL של הקבצים במערכת :

F 1	**U1: rw , U2: rwx**
F 2	**U1: w, U3: rw**
F 3	**U1: rx, U3: rw**

האם הנתונים מתוך קובץ F2 יכולים להגיע(בדרך ישירה או עקיפה) לידי משתמשים U1 ו U2 ?

א. לא, לאף אחד מהם.

ב. כן, רק ל U1 .

ג. כן, רק ל U2 .

ד. כן, גם ל U1 וגם U2 .

שאלה 8

מה נכון לגבי HYPERVISOR מסוג 1 בתצורת bare metal?

א. כל משאבי החומרה hardware resources נשלטים **באופן ישיר** ע"י hypervisor .

ב. רק הקצאות זיכרון ראשי ופעולות קלט-פלט עוברות דרך hypervisor , אבל הקצאת זמן המעבד CPU הפיזי מבוצעת **ישירות** ע"י ה OS האורחת **בלי שיתוף** של hypervisor .

ג. רק הקצאות זיכרון ראשי עוברות דרך hypervisor , אבל פעולות קלט-פלט מבוצעות **ישירות** ע"י ה OS האורחת **בלי שיתוף** של hypervisor .

ד. כאשר יישום המשתמש מבצע את system calls ל-OS שרצה על VMM , אותה ה OS **לא תנסה** לבצע את ה- system calls ב kernel mode .

המשך הבחינה בעמוד הבא

לא לכתוב בשוליים

נא לא לכתוב בשוליים

שאלה 9

בחרו מידע אשר **איננו מצוי** ב PCB (process control block) של תהליך :

א. מצב של תהליך (ריצה, המתנה וכו׳).
ב. מצביע לטבלת קבצים פתוחים אשר נפתחו ע״י התהליך.
ג. מצביע למפת הזיכרון(טבלת הדפים) של התהליך.
ד. מונה מספר בלוקים שנקראו מדיסק ע״י התהליך.

שאלה 10

מהו החיסרון המובהק של שיטת ה- rendezvous(שיטת המפגש) לעומת ה- mailbox ב- message passing?

א. שיטת ה-rendezvous קשה יותר למימוש במערכת הפעלה.
ב. שיטת ה-rendezvous פחות גמישה מכיוון שהתהליכים (השולח והנמען) הופכים להיות תלויים זה בזה.
ג. בשיטת ה-rendezvous לא ניתן לממש מניעה הדדית בגישה לקטעים קריטיים (critical sections).
ד. שיטת ה-rendezvous ניתנת למימוש רק אם מעבד CPU הוא בעל ליבה CORE אחת.

שאלה 11

מה מבין המפורטות היא הדרך הנכונה **והסבירה** למניעת deadlock במערכת עם 4 RESOURSES ?

א. כל process ישחרר את כל RESOURCES שהוא **החזיק** בהם(כולל אמצעי סנכרון) לפני שיסתיים.
ב. כל process יוכל לבקש לא יותר מ 2 RESOURCES .
ג. כל ה processes יוכלו לבקש ולהחזיר רק RESOURSE אחד בכל נקודת זמן של חיי התהליך.
ד. כל ה processes יוכלו לקבל את RESOURCES רק לפי סדר עולה ולשחרר לפי סדר יורד.

המשך הבחינה בעמוד הבא

$20\cdot 2^{10}$ = 10 בלוקים - נתונים. סה״כ 20KB נתונים

בלוק indirect יצביע ל: $\frac{2KB}{4B} = \frac{2^{11}}{2^2} = 2^9$ בלוקים 1503

כלומר $2^9 \cdot 2KB = 2^9 \cdot 2^{11}$

עוד $1503-2^9=$ ~~1503~~ $-$

בלוק double indirect: ~~יצביע ל 2^9 בלוקים~~ 991

990

מצביע 1 בו יצביע לבלוק המכיל 2^9 מצביעים לבלוקים.

(סה״כ $1+2^9$ בלוקים ו $2KB\cdot 2^9$ נתונים)

(נשאר בדיוק 477

עוד מצביע בו יצביע לבלוק המכיל 2^9

מצביעים לבלוקים אבל הפעם נשתמש

רק ב 476 מהם

(סה״כ 477 בלוקים ו $476\cdot 2KB$ נתונים)

סה״כ ~~נתונים~~:

מס׳ בלוקים של מידע: $10+2^9+2^9+476=1510$

מס׳ בלוקים סה״כ מדרגה: 1 single indirect

1 לרמה עליונה של double

2 לרמה התחתונה של double

סה״כ נתונים: $1510\cdot 2KB=3020KB$

נא לא לכתוב בשוליים

שאלה 12

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node עם פרמטרים הבאים :

- גודל הבלוק במערכת הקבצים הוא 2 Kbytes
- כתובת הבלוק היא 4 בתים (bytes)
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block

חשבו מה הגודל המרבי של קובץ שניתן לאכסן ב 1514 בלוקים בסה"כ(כולל נתונים ומצביעים, אבל לא כולל את ה I-NODE עצמו) .

הערה : לצורך החישוב 1 Kbyte = 1024 Byte , 1 MByte = 1024 Kbyte

א. 3028 Kbytes

ב. 3020 Kbytes

ג. 3000 Kbytes

ד. 3008 Kbytes

בהצלחה!

שאלון 517 82.81.7 M4

נא לא לכתוב בשוליים